

David Souther

Brooklyn, New York davidsouther+resume@gmail.com · (650) 495-5402 davidsouther.com · github.com/davidsouther

RESEARCH AND TEACHING INTERESTS

Computing education for domain experts. Nineteen years of industrial software engineering alongside a decade of teaching, with a consistent focus on teaching domain experts with little to no programming exposure to the point where they can build, deploy, operate, and troubleshoot real software in their own field. This extends to the subject of my current graduate research, approaches to teaching software engineering to mid-career professionals with little to no programming experience.

Auditable AI-assisted software engineering. Agentic large-language-model systems can absorb substantial portions of the software development lifecycle, but only where human guidance in the form of prompts, context, and generated output are durable, reviewable, artifacts guided with an understanding of the tooling in the holistic context of software engineering. My open-source work and recent writing pursue this effort directly.

Research software, analytics, and visualization. Building the data, service, and interface layers that let a research group ask questions of large, heterogeneous datasets — and rendering the answers so a non-specialist users can act on them.

The software development lifecycle as collaborative practice. Engineers, program and project managers, interface designers, subject-matter experts, and end users each take part in a working system. Teaching that collaboration is as much of the work as teaching the practice of programming.

EDUCATION

M.S., Computer Science — Brooklyn College, City University of New York *Spring 2026 – present. Enrolled, in good standing.* Research focus: pedagogy of software engineering for mid-career professionals transitioning to software engineering without a programming background.

B.S., Mathematics — Rocky Mountain College, Billings, MT · 2011 Coursework and independent study in calculus and discrete mathematics. Tutored mathematics for three years. Organized a standing study group among mathematics majors that worked through coursework alongside the broader ideas — philosophy, religion, and science — connecting to the material.

B.S., Computer Science — Rocky Mountain College, Billings, MT · 2011 Two Bachelor of Science degrees completed in five years. Collaborated with the computer science department on software projects for other research groups on campus, including tooling that let **computational biology students run genome analyses on Rocky Mountain College's local computing cluster** — the first instance of a pattern that has recurred through my career: building the computational apparatus that lets domain researchers do their own science.

ACADEMIC AND TEACHING APPOINTMENTS

Lead Instructor — Code Fellows · March 2022 – June 2023 Lead instructor across the code fellows curriculum, an intensive career-change program for adult learners. Responsible for classroom instruction, curriculum development, student evaluation, and

career coaching. Courses consistently earned **above an 80 net promoter score** on both weekly surveys and end-of-course evaluations. Organized and led an instructor-wide project rebuilding the school's technical-whiteboarding process, spanning a rewritten guide, a formal grading rubric for whiteboards and technical interviews, and revised problem and training materials for a dozen instructors across four programming languages. Code Fellows was named a top coding bootcamp by *Fortune Education* in 2023.

Lecturer — Chatham University, Pittsburgh, PA · August 2016 – May 2017 Taught an undergraduate business course on the effects of technology in the 21st century, surveying technological topics at their intersection with business ethics. Students learned to apply computing to everyday professional needs while examining the implications of large-scale data collection, always-on connectivity, and the largely invisible ways engineering tools shape daily life. **A technical course taught to non-technical undergraduates, in a non-technical department.**

Instructor — Mount Holyoke College and Smith College, in partnership with Google · November 2015 – January 2017 Developed and taught January-term intensive courses in mobile software development and team software engineering. Materials covered test-driven development in Python and Android development on physical devices, centering a substantial team project component. Delivered with colleagues from Google as an industry-academic partnership.

Instructor (industry training) — ProTech, via Third Cat LLC · 2014 – 2015 JavaScript and web development trainer for a national IT training provider. Delivered on-site instruction to professional development teams adopting the Node.js and AngularJS stack.

Internal instruction — Google · 2015 – 2017 Taught internal courses on TypeScript and Angular to engineering teams, and delivered a two-week intensive on test-driven development and industry practices at Mount Holyoke College. Taught interview training courses for Google interviewers, and technical interviewing courses for prospective candidates and collegiate students.

CURRICULUM DEVELOPMENT

Code 401 curriculum — Code Fellows, 2022–2023. Course material for a full-stack TypeScript program serving adult career-changers. Materials focused on the range of full stack engineering including storage systems, api design, data structures & algorithms, HTML & React interfaces, user experience design, and collaborative software engineering tools.

Technical whiteboarding curriculum and rubric — Code Fellows, 2022–2023. Rewrote the institutional whiteboarding guide, formalized an assessment rubric for whiteboard and technical-interview evaluation, and revised problem sets and instructor training materials for approximately twelve instructors across four languages. Published openly; see *Publications*.

Software Craftsmanship for the Lay Person — 2013–present. A project-based introductory book for a first exposure to programming. The main text is language-agnostic; three companion workbooks give project-specific instruction in Python, TypeScript, and Rust. github.com/DavidSouther/software_craftsmanship

nand2tetris Web IDE — 2021–present. Primary contributor to the browser-based IDE for the *nand2tetris* computer architecture and language course, removing local toolchain installation as a barrier to entry for students & generally modernizing the user interface. github.com/nand2tetris/web-ide

Code Explainer — AWS TCX AI Lab, 2024. Prompt-engineered tooling generating explanations of example code, targeted to defined AWS learner levels — instructional scaffolding calibrated to a reader's stated expertise.

PROFESSIONAL APPOINTMENTS

Staff Software Engineer — Nominal · May 2026 – present Nominal Connect is a desktop application through which test-stand engineers acquire, view, and control hardware telemetry, automate test-stand procedures, and stream the resulting data to a shared platform for offline processing. I joined to build the assistant layer: in-application help search, deep stateful project search, and application-state awareness. In connected modes, LLM-backed search supports free-form natural-language querying, and engineers can make natural-language edits to their own applications — **removing substantial engineering workload and improving reliability for domain experts performing software tasks they were never trained for.** Ground up design, implementation, and deployment of a complete information retrieval system, including hybrid lexical and semantic search ranking. Rust, Bevy, agentic AI.

Staff Software Engineer — Apollo GraphQL · December 2025 – April 2026 Apollo's MCP Gateway is a core network component for enterprise Model Context Protocol management, brokering all MCP sessions in a customer environment while enforcing authentication and organizational policy. I designed the runtime components, enabling clean deployment across environments from local development through Docker Compose to full Kubernetes operator orchestration. I championed an organizational pivot from direct control-plane-to-gateway configuration toward an infrastructure-as-code model that grows with an organization from click-ops to git-ops, and implemented portions of that and other designs in the Rust codebase. Mentored engineers new to Rust across the company, emphasizing composable design patterns that work with the grain of both the language and the application.

Senior Software Development Engineer (SDE III / L6) — Amazon Web Services, Technical Content Experience · September 2022 – November 2025

Technical Lead, Content Automations (December 2024 – November 2025). Led the build process producing **more than 50,000 code snippets across more than 5,000 examples for 15 core AWS SDK technologies**, embedded throughout AWS technical documentation and the primary point of contact many engineers have with learning AWS SDK programming. The system coordinates contributions from dozens of developers and hundreds of writers to keep examples accurate and current, through extensive CI automation, internal and external content treatments, and stakeholder dashboards. Separately led *IAM Modernization*, a cross-functional large-scale code migration for a technical writing organization: extracted, verified, and corrected **12,000 IAM policies across 250 technical guides in a single quarter**, reducing an estimated 4,000 hours of labor to a logged 1,800, and leaving a reusable framework for future migrations.

AI Lab (January 2024 – December 2024). Member of an organization-wide cross-functional group investigating applications of AI — particularly large language models — to authoring and educational workflows. The lab workshopped many applications, moved several to production, and ran controlled experiments to validate others. My contributions included Code Explainer (learner-level-targeted code explanation) and a Style Guide Checker experiment applying prompt engineering to identify and correct both general grammatical issues and organization-specific content guidance.

Code Examples (September 2022 – November 2024). Senior engineer creating example code and applications for the AWS SDK for Rust, with longer-horizon work on cross-service scenarios demonstrating real-world software combining multiple AWS services. Selected work: a REST system built on Amazon RDS Data and Amazon SES; a serverless photo asset manager using S3 intelligent tiering, Amazon Rekognition image tagging, and SQS notifications; and a bespoke human-navigator LLM workflow built with Ailly and Amazon Bedrock that **accelerated team development by 20%**.

Senior Software Engineer (L3) — SpaceX, Starlink · July 2021 – June 2022 Responsible Engineer for a global ISP's ground-network off-premises cloud tooling, including global data acquisition, Data Center Infrastructure Management, and automated authentication; migrated and operated these systems across VM and service mesh environments. Responsible Engineer and subject-matter expert for a DDoS detection, alerting, and mitigation system protecting core infrastructure and millions of customers. Responsible Engineer for cloud spend, migrating roughly 30% of workload on-premises and reducing overall utilization by about 10% while the user base grew approximately 50%.

Senior Software Engineer (L5) — Google, Google Cloud Platform · November 2018 – July 2021

Compute Front End (July 2020 – July 2021). Technical lead setting strategic direction for the feature area; led a team of ten (four direct reports) migrating a cloud management tool from AngularJS to Angular while sustaining feature delivery to maintain market position through the migration. As a manager, developed junior and newly hired engineers in Cloud, Angular, and TypeScript.

Cloud Topology (November 2018 – June 2020). Cloud Topology lets GCP customers visualize large-scale deployments. Built a **Kubernetes visualization rendering 10,000 nodes at 60fps with hierarchical graph layout in under one second**, through a purpose-built high-performance graph rendering engine and direct research with internal and external users. Designed a graphical data model and visualization improving situational awareness of cluster communication patterns, validated by early-adopter teams reporting improved deployment decisions. Built and documented a large-scale Kubernetes test bed enabling 10,000-node cluster testing for internal teams.

Software Engineer (L4) — Google, Google Shopping · June 2015 – April 2017 Front-end engineer with Channel Intelligence. Led development and served as architect of the core platform and tooling for a user interface managing product-level experiments on shopping data, including complex workflows for constructing experimental hypotheses, treatments, and groupings. Contributed to a cross-working-group migration from GWT web UIs to component-based Angular architectures. Advocated and taught test-driven development practice across the front-end organization.

Owner and Founder — Third Cat LLC · June 2014 – July 2015 Consulting firm specializing in JavaScript technologies. Architected and built a scalable real-time monitoring platform for Data Online, a New Jersey manufacturing firm, and trained their team in single-page application development. Provided visualizations for a DartmouthX online engineering course. Long-term engagement with ProTech as a JavaScript and web development trainer.

Software Engineer, Head of Front End Development — Novus Partners Inc · October 2013 – June 2014 Led a team of four through a complete front-end rewrite, decoupling a brittle Scala Server Pages implementation into an AngularJS front end and a focused

Scala backend. Built and maintained the **NVD3 open-source charting library**, leading a sustained effort to refactor the original prototype into a maintainable, extensible, testable library for front-end data visualization.

Associate Software Engineer — The New York Times · July 2012 – October 2013 Full-stack developer on the NYT5 rebuild team, reconstructing nytimes.com from the ground up — a property serving 45 million unique monthly visitors at the time. Core member of the Prototype team building and testing candidate features on proto.nytimes.com. Responsible for the backend stack, including Node servers for data management and A/B testing, and supporting systems administration. Invited to speak on emerging technologies to internal development groups and, on behalf of the Times, at regional college technology events and meetups.

Software Engineer — Potomac Fusion, Inc · September 2011 – June 2012 Built Synapse, a government open-source tool bringing large-scale data handling and visualization into the browser, sharing tens to hundreds of thousands of records across discrete widgets in a shared web desktop. The widgets gave analysts visualization and tooling to work across disparate streaming data sources and act on what they found.

Software Architect — Design Delegates · October 2010 – July 2011 Responsible for the full technology stack at a startup software development company. Built platforms to facilitate web programming, implemented continuous integration across several languages and frameworks, and architected medium-scale projects with a team of five developers and designers. The largest project was a field-agent tracking system maintaining scheduling and routing for hundreds of in-store marketing agents in Australia, combining a scalable PHP backend, a dynamic mobile front end, and extensive reporting. In the course of that work we developed JEFRI, a JavaScript entity framework.

Software Developer — Entre Technology Services, LLC · September 2007 – September 2010 Managed medium-scale software projects for Montana companies including Stillwater Mining Company, Town Pump of Montana, and Crowley Fleck Law Firm. Developed a solution to e-discovery motions, wrote OSHA tracking software, built modules for inventory management systems, and supported web development for several clients.

MENTORSHIP AND ADVISING

Engineering management — Google Cloud Platform, 2020–2021. Directly managed and developed junior and newly hired engineers. Promoted an L4 to L5 in three performance cycles and an L3 to L4 in two, and guided an engineer through a performance improvement plan who was subsequently promoted two cycles later. This work comprised technical mentorship, performance review and promotion packet authorship, and coordination of priorities and personnel concerns.

Technical mentorship — Apollo GraphQL, 2025–2026. Mentored engineers new to Rust across the company, focused on finding composable design patterns that work with the grain of both the language and the problem domain.

Career coaching — Code Fellows, 2022–2023. Career coaching and student evaluation as a core instructional responsibility for adult career-changers.

Undergraduate tutoring — Rocky Mountain College, 2008–2011. Three years tutoring mathematics; convened and sustained a peer study group among mathematics majors.

PUBLICATIONS AND PUBLIC SCHOLARSHIP

Manuscripts in Preparation

Souther, D. "LLMs as a Model of Syntactic Space: The Document Manifold and a Lens for Agentic Workflows." *In preparation*. A synthesis and position paper framing large language models as machine-learned trajectories over the syntactic space of language, and exploiting that frame as a design recipe for agentic AI workflows.

Defensive Publication

Souther, D., et al. "Graph Visualization by Organizing Connections in Collapsible Hierarchical Graphs." *Technical Disclosure Commons*, Defensive Publications Series, Art. 2996, 8 March 2020. First author. tdcommons.org/dpubs_series/2996 · Full text (PDF)

Network graphs in domains such as cloud networking carry connections across multiple dimensions — communication and configuration — producing a compound graph in which every node may contain a complete graph beneath it. Prior layout algorithms treat the hierarchy as a hinting constraint and lay out only the leaves, yielding a single static image of considerable complexity in which expanding a node can drastically rearrange the result. The disclosure defines a *hull* as a node together with its descendants and a *segment* as a bundle of edges between the descendants of a pair of nodes. Layout proceeds over the top level rather than the leaves; expansion changes only a node's size, never its position; and edges route up the ancestor chain, across once at the top level, and back down to their targets. Expansion state is tracked in the graph structure itself, so all visibility decisions resolve at render time. The result is a stable high-level visualization of a large graph network that can be refocused to detail without perturbing the layout. This is the formal write-up of the rendering approach behind the Cloud Topology work described above.

Essays on Computing Education

Souther, D. "Technical Whiteboarding." Series, 2024. A systematic approach to designing algorithmic solutions to problems encountered in software development. Best known from technical interviewing, but the method applies wherever there is a problem to solve with code.

- Part 1: Technical Whiteboarding
- Part 2: Drawing Data Structures
- Part 3: Tracing Algorithms
- Part 4: The Forward/Backward Method for Algorithms
- Part 6: Whiteboarding Glossary

Souther, D. "Forward/Backward Method." 2026. Generalizes the algorithmic technique into a planning pattern for traversing unknown intermediate terrain between a known start and a known goal.

Souther, D. "-ilities: Functional and Non-Functional Requirements." 2023. Transcript of a lecture delivered to a Code Fellows 401 JavaScript course, on how formalized non-functional requirements let a system grow sustainably across its product lifecycle.

Souther, D. "Behavioral Interviewing." 2023. On the STAR narrative format and the structure of behavioral evaluation.

Souther, D. "Technical Whiteboarding" (curriculum). Code Fellows common curriculum, 2023. Institutional guide covering the checklist for working through a technical problem, methods for diagramming programs, an extensive data structures and algorithms glossary, and the Forward/Backward Method applying mathematical proof technique to systematic problem solving. codefellows.github.io/common_curriculum

Essays on Large Language Models

Souther, D. "Jeopardy! Search." 2026. A semantic search technique that inverts document expansion: rather than generating a hypothetical document similar to source documents, it generates several hypothetical *queries* a document would answer, moving semantic expansion into query space. Multiple queries are run in parallel to cover a wider range of the document space, and converts hallucinations from a liability into an asset.

Souther, D. "Ailly OODA." 2026. Applies the Observe–Orient–Decide–Act framework to collaborative coding agents that work alongside a developer rather than replacing them, including the role of an explicit hypothesis attached to each decision.

Souther, D. "LLM Review is not Human Review." 2026. On the distinct aims of machine and human review: LLM review finds patterns a human may have missed, but pulls output toward an average, while human review is a two-way meeting of minds.

Souther, D. "Beyond Knowledge–That: LLMs' Indirect Understanding." 2024. Applies Russell's distinction between knowledge by acquaintance and knowledge by description to resolve the apparent contradiction in saying a model both does and does not know something.

Souther, D. "Fuzzy Homomorphic Endofunctors." 2024. LLMs as exploratory fuzzy homomorphic endofunctors mapping between points on the manifold of syntactically valid documents. The informal seed of the manuscript in preparation above.

Essays on Software Architecture

Souther, D. "Arrow of Maturity." 2025. On the common path along which projects grow architecturally — prototyping, straight-through handlers, three phases of domain-driven design, event-sourced microservices — and how to keep an architecture sustainable across that lifetime. Opens on the observation that most real-world computation begins as spreadsheet models built by non-programmers.

Souther, D. "Notes on Roy Fielding's REST Dissertation." 2021. A close reading of the dissertation, paired with a twenty-years-later reply applying lessons from releasing, managing, and consuming resource-oriented APIs at cloud scale.

Industry Writing

Souther, D., et al. "Cloud Journeys: Building a Serverless Image Recognition Website with Machine Learning." *community.aws*, June 2023. Inaugural entry in the Cloud Journeys content series, narrating the construction of a serverless application that detects labels for images and lets users retrieve images by label.

Community

Stack Overflow contributor. Ranked in the **top 5%** of answerers in AngularJS, debugging, Grunt.js, JavaScript, Node.js, and PHP; **top 10%** in Bash, C, HTML, and Java; **top 20%** in Android and Linux. Widely referenced answers include "Is stat() an expensive system call?" (2013) and "Format Date time in AngularJS" (2012).

SOFTWARE AND OPEN SOURCE

Ailly — 2023–present. An LLM authoring and agentic-development tool built around a simple premise: prompts live as snippets on the file system, so prompt engineering can be iterated at fine grain and tracked with ordinary source control. Both the author's changes and the model's outputs become reviewable, diffable history. **This is the mechanism by which LLM-assisted work becomes auditable** rather than ephemeral, and it underpins my interest in bringing agentic development into a research setting responsibly.

nand2tetris/web-ide — 2021–present. Browser-based IDE for the nand2tetris computer architecture and language course.

Jiffies CSS — 2021–present. A "postmodern" CSS full-page reset built on current pure-CSS standards — cascade layers, native nesting, and a variable structure supporting user and application overrides.

Software Craftsmanship for the Lay Person — 2013–2021. Project-based introductory programming book with Python, TypeScript, and Rust workbooks.

Montana News Archive — 2015–2017. Long-term archival and search tooling for local broadcasters, used by Montana and regional news networks to incorporate historical footage into broadcasts.

NVD3 — 2013–2014. Maintainer of the open-source D3-based charting library during its refactor from prototype into a maintainable, testable library.

JEFRi — 2011. JavaScript entity framework developed to support mobile application development.

INVITED TALKS AND PRESENTATIONS

"-ilities: Functional and Non-Functional Requirements." Lecture delivered to a Code Fellows Code 401 JavaScript course; transcript published August 2023. See *Publications*.

Emerging technologies. Invited talks to internal development groups at The New York Times, and, representing the Times, at regional college technology events and technology meetups, 2012–2013.

Test-driven development and industry practices. Two-week intensive, Mount Holyoke College, delivered with Google colleagues, 2015–2017.

SERVICE

Instructional process leadership — Code Fellows, 2022–2023. Convened and led an instructor-wide working group that rebuilt the institution's technical assessment process, delivering a rewritten guide, a formal grading rubric, and revised training materials adopted by approximately twelve instructors across four programming languages.

Open-source maintenance — Ongoing contribution and maintenance across the projects listed above, including the nand2tetris Web IDE, which serves students of a widely adopted computer architecture curriculum.

TECHNICAL COMPETENCIES

Languages — Rust, Python, TypeScript, JavaScript, Java, Go, C, C++, Scala, PHP, Dart, SQL, Bash

Machine learning and AI — Large language models, prompt engineering, agentic systems, Model Context Protocol, Amazon Bedrock, retrieval and semantic search, LLM evaluation harnesses

Data, analytics, and visualization — Large-scale graph rendering and layout, D3, SVG, NVD3, data pipeline design, dashboard and reporting systems

Systems and infrastructure — Kubernetes, Istio, service mesh, Docker, AWS, Google Cloud Platform, distributed systems, CI/CD automation, Linux, Node.js, GraphQL

Practice — Test-driven development, software development lifecycle facilitation, technical writing and documentation systems, curriculum design, code review, technical interviewing and assessment, team leadership and people management

References available on request.